SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences, Chennai – 602105

A CAPSTONE PROJECT REPORT

Advanced Banking and Financial Transaction Management System with Scheduled Payment Automation

Module 3 – Scheduled Payment Automation Module

A Capstone Project Report submitted in partial fulfilment of the requirements for the Course of

DSA0112 – OBJECT-ORIENTED PROGRAMMING C++

towards the award of the degree of

BACHELOR OF TECHNOLOGY IN

INFORMATION TECHNOLOGY / COMPUTER-RELATED BRANCH

Submitted by

KRITHIYA G – 192421150

Under the Supervision of

DR.SUDHA I

SEPTEMBER

BONAFIDE CERTIFICATE

Saveetha Institute of Medical and Technical Sciences

Chennai – 602105

This is to certify that the Capstone Project entitled "Advanced Banking and Financial Transaction Management System with Scheduled Payment Automation – Module 3 – Scheduled Payment Automation Module" has been carried out by Krithiya -192421150 , Shruthi sree - 192421129 and Patan Aufrin -192473029 under the supervision of [Guide Name] and is submitted in partial fulfilment of the requirements for the current semester of the [Course Name] programme at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Dr. C. Sivasankar

Professor

Department Name of CSE

Saveetha School of Engineering,

SIMATS, Chennai – 602105.

COURSE FACULTY

Dr. I. Sudha

Professor

Department Name of CSE

Saveetha School of Engineering,

SIMATS, Chennai – 602105.

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

DECLARATION

I Krithiya G of the Oject - oritented programming c++ programme, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled "Advanced Banking and Financial Transaction Management System with Scheduled Payment Automation – Module 3 – Scheduled Payment Automation Module" is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with engineering ethics and academic integrity. All sources, references, datasets, software resources, and other materials used in the project have been appropriately acknowledged. We further declare that the analysis and findings presented in this report have not been fabricated, falsified, or manipulated.

Place: Chennai

Date: _____

Signature of the Students with Names

INDIVIDUAL CONTRIBUTION STATEMENT

Mandatory for all team projects

Module 3 – Scheduled Payment Automation Module

The Advanced Banking and Financial Transaction Management System is a modular platform designed to manage customer accounts, financial transactions, and automated payment processing. Module 3 focuses specifically on Scheduled Payment Automation — enabling customers to set up one-time and recurring payments (such as bill payments, fund transfers, EMIs, and subscription debits)

Student Name / Register No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
Aufrin 192473029	**SECURE USER AUTHENTICATION & ACCOUNT ACCESS**	**Designed the scheduled payment module and implemented recurrence options for one-time, daily, weekly and monthly payments**	**Tested different scheduling intervals and payment dates**	**Introduction and problem statement**	**25%**
Shruthi sree 192421129	**BANKING OPERATIONS & SECURE TRANSACTION MANAGEMENT**	**Developed validation modules for account, beneficiary and payment information**	**Tested different account details, insufficient balance conditions and beneficiary inputs**	**System design and methodology**	**25%**
Krithiya 192421150	**SCHEDULED PAYMENT AUTOMATION MODULE**	**Developed execution and retry modules using retry-with-backoff and integrated them with the transaction module**	**Checked successful and failed payments and tested different retry conditions**	**Testing and results section**	**25%**
Total					**100%**

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

Symbol	Description	Unit / Context
API	Application Programming Interface	Data access
DB	Database	Data storage
EMI	Equated Monthly Instalment	Loan repayment
JSON	JavaScript Object Notation	Data format
OTP	One-Time Password	Authentication
REST	Representational State Transfer	API architecture
SI	Standing Instruction	Recurring payment
SLA	Service Level Agreement	Reliability target
SMS	Short Message Service	Notification channel
UI	User Interface	Presentation layer

CHAPTER 1
INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

The Advanced Banking and Financial Transaction Management System is a multi-module platform that manages customer accounts, processes financial transactions, and automates routine banking operations. Module 1 handles account and customer management, and Module 2 handles core transaction processing and the ledger. Module 3 – Scheduled Payment Automation – builds on these foundations to allow customers to configure one-time or recurring payments that are automatically validated and executed on their due dates without manual intervention.

Scheduled Payment Automation covers use cases such as recurring bill payments, EMI debits, subscription payments, and standing instructions for fund transfers. The module is responsible for storing schedule definitions, triggering execution at the correct time, validating funds and beneficiary details, invoking the core transaction engine, handling failures with retries, and keeping the customer informed through notifications.

1.2 Need for the Project

- Manual bill payment and fund-transfer reminders are time-consuming and prone to human error, often resulting in missed due dates and late-payment penalties.
- Customers with recurring financial obligations (EMIs, utility bills, subscriptions) benefit from automated, dependable execution rather than repeated manual entry.
- Existing manual processes provide no structured retry or failure-notification mechanism when a scheduled payment cannot be completed due to insufficient balance.
- Banks require an auditable record of scheduled and executed payments to satisfy compliance and customer-dispute resolution needs.

- An automation layer is required to connect account management and transaction processing with reliable, on-time, and traceable execution of recurring payments.

1.3 Problem Statement

The engineering problem is to design a Scheduled Payment Automation module that allows customers to create, modify, pause, or cancel one-time and recurring payment instructions; automatically triggers execution of due payments at the correct date and time; validates account balance and beneficiary details before executing a transaction; retries failed executions according to a defined policy; and notifies the customer of the outcome, while maintaining a complete, auditable log of all scheduling and execution activity.

1.4 Project Aim

To develop Module 3 of the Advanced Banking and Financial Transaction Management System for automated scheduling, validation, and execution of one-time and recurring financial transactions.

1.5 Project Objectives

- To allow customers to create one-time and recurring scheduled payments with configurable frequency (daily, weekly, monthly, or custom intervals).
- To design a scheduler/trigger service that identifies due payments and initiates execution at the correct date and time.
- To validate account balance, account status, and beneficiary details before executing a scheduled transaction.
- To integrate with the core transaction and ledger module for posting successfully validated payments.
- To implement retry-with-backoff and customer notification for failed or delayed executions.
- To maintain a complete, tamper-evident audit log of all scheduled and executed payments.
- To allow customers to view, edit, pause, resume, or cancel scheduled payments through the user interface.

1.6 Scope

Included: creation, modification, pause, resume, and cancellation of scheduled payments; a recurrence-rule engine; a scheduler/trigger service; pre-execution validation; retry and failure handling; customer notification; and an audit log with a scheduled-payments dashboard.

Excluded / Boundary: the module does not perform core account creation or KYC (handled by Module 1), core ledger posting logic (handled by Module 2), or external payment-gateway / inter-bank settlement processing, which are treated as downstream dependencies.

1.7 Expected Engineering Outcome

The expected outcome is a functional scheduling-and-automation component that reliably triggers, validates, and executes one-time and recurring payments, forwards successful transactions to the core ledger, retries or reports failures, and provides customers and administrators with visibility into scheduled-payment status through an auditable record.

CHAPTER 2
LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

Existing approaches to recurring-payment automation were reviewed across three areas: (a) banking standing-instruction and autopay features offered by conventional core-banking systems, (b) subscription-billing platforms that trigger recurring charges (for example, card-based recurring billing engines), and (c) general-purpose job-scheduling and event-driven architectures (cron-style schedulers, message queues, and workflow engines) that are commonly used to build reliable, time-triggered automation.

2.2 Review of Existing Work

Conventional core-banking systems typically offer standing-instruction features that transfer a fixed amount on a fixed date, but they often provide limited visibility into failure reasons and limited retry configurability for the customer. Subscription-billing platforms emphasise idempotent, retry-driven execution to avoid duplicate charges and commonly use exponential backoff for failed attempts. General-purpose job-scheduling and message-queue architectures provide reliable trigger mechanisms and are widely used as the underlying engine for time-based automation, but they need to be adapted with domain-specific validation, ledger integration, and compliance logging to be suitable for a banking context.

2.3 Comparative Analysis

Compared with a purely manual, reminder-based approach, automated scheduling systems substantially reduce missed payments and repetitive data entry. Compared with generic job-scheduling frameworks used in isolation, a banking-specific scheduled-payment module must additionally provide balance validation, beneficiary verification, idempotent execution (to guarantee a payment is not posted twice), and a compliance-grade audit trail — requirements that are not addressed by generic schedulers out of the box.

2.4 Research / Engineering Gap

The reviewed approaches show that reliable time-based triggering is a solved problem in general software engineering, and that recurring billing engines

address idempotency and retries. However, integrating these techniques with core-banking validation (balance checks, beneficiary verification, ledger posting) and providing customer-facing controls (pause, resume, cancel) and transparent failure notification within a single cohesive module remains an area that individual banking applications implement inconsistently. Module 3 addresses this gap for the Advanced Banking and Financial Transaction Management System.

2.5 Proposed Contribution

The proposed contribution is an integrated Scheduled Payment Automation layer that combines a configurable recurrence engine, a reliable trigger/scheduler service, pre-execution validation against the account-management module, idempotent execution through the core transaction module, retry-with-backoff on failure, customer notification, and a complete audit log — all exposed through a customer-facing dashboard for creating and managing scheduled payments.

CHAPTER 3
ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User	Requirement
Customer	Create, edit, pause, resume, and cancel scheduled payments through the interface.
Customer	Receive timely notification of successful or failed scheduled-payment execution.
System	Accept validated account and beneficiary details from Module 1.
System	Trigger execution of due payments automatically at the scheduled date and time.
System	Retry failed executions according to a defined retry policy.
System	Maintain an auditable log of all scheduling and execution activity.

Requirement Type	Measurable Target
Functional	A scheduled payment can be created with a one-time or recurring frequency.
Functional	Due payments are automatically detected and executed by the scheduler.
Functional	Failed executions are retried up to a configured maximum before being reported to the customer.
Non-Functional	Scheduled payments should be triggered within a defined time window of the due time (target: within 5 minutes).
Non-Functional	The module should integrate consistently with Module 1 (accounts) and Module 2 (transactions).

3.2 Constraints & Applicable Standards

The module must comply with the bank's transaction-authorisation and audit-logging policies and must not post a duplicate debit for a single scheduled execution (idempotency). Practical constraints include dependency on the accuracy and availability of the account-management and core-transaction modules, clock/time-zone synchronisation for trigger accuracy, and the need to fail safely (i.e., decline execution rather than risk an incorrect debit) when validation cannot be completed.

3.3 Design Specifications

Parameter	Target / Requirement	Unit / Context	Priority
Schedule input	Payee/beneficiary, amount, frequency, start date, end condition	Text / Date	High
Recurrence	Daily, weekly, monthly, or custom interval	Rule set	High
Trigger accuracy	Execution initiated close to scheduled time	Minutes	High
Validation	Sufficient balance and valid beneficiary before execution	Boolean check	High
Retry policy	Automatic retry on failure with backoff	Attempts / interval	Medium
Audit log	Immutable record of scheduling and execution events	Log entries	High
Notification	Customer alert on success or failure	SMS / Email / App	Medium

3.4 Alternative Solutions & Evaluation

Alternative 1: Manual reminder-based payment. The customer is reminded of an upcoming due date and must manually initiate the payment.

Alternative 2: Internal scheduler and execution engine integrated with the core ledger. A dedicated scheduler service triggers validation and execution through the existing transaction module.

Alternative 3: Third-party autopay / payment-gateway subscription service. Recurring debits are delegated to an external payment gateway.

Criterion	Weight	Alt. 1 Manual Reminder	Alt. 2 Internal Scheduler + Ledger Integration	Alt. 3 Third-Party Autopay Gateway
Performance	High	Low	High	Medium
Cost	Medium	Low	Medium	Medium
Reliability	High	Low	High	Medium
Auditability	High	Low	High	Medium
Maintainability	Medium	High user effort	High with automation logic	Requires gateway dependency
Selection			Selected	

3.5 Selected Design & Detailed Design

The internal scheduler-and-execution-engine approach (Alternative 2) is selected because it keeps the recurring-payment logic, validation, and audit trail fully within the bank's own system, avoiding dependency on an external gateway for core banking obligations while still allowing gateway integration for external payees where required.

Processing Flow: Schedule Creation → Recurrence Rule Stored → Scheduler Detects Due Payment → Balance & Beneficiary Validation → Execution via Core Transaction Module → Success: Ledger Update + Notification + Audit Log; Failure: Retry (with backoff) → Notification + Audit Log.

Key design elements include a recurrence-rule engine (supporting daily, weekly, monthly, and custom intervals), an idempotency key per scheduled execution to prevent duplicate posting, a configurable retry policy with exponential backoff, and a customer dashboard for viewing and managing scheduled payments.

3.6 Trade-off Analysis

Reliability versus latency: more frequent scheduler polling increases trigger accuracy but adds system load, while event-driven triggering reduces load but adds implementation complexity. Retry attempts versus duplicate-transaction risk: additional retries improve the chance of eventual success but require strict idempotency controls to avoid double debits. Centralised internal execution (chosen design) trades some flexibility of third-party gateway features for tighter control, auditability, and consistency with the core ledger.

3.7 Sustainability, Ethics, Safety, Risk & Security

Ethics and transparency: customers must explicitly consent to each recurring-payment instruction and must be able to view, pause, or cancel it at any time; all execution outcomes are communicated clearly. Security: stored payment instructions (beneficiary and amount details) are encrypted at rest, and access to scheduling data is restricted through role-based access control. Safety: the system is designed to fail safely — if validation cannot be completed with confidence, the scheduled execution is deferred and reported rather than forced through.

Risk	Likelihood	Impact	Mitigation
Insufficient balance at execution time	Medium	Medium	Pre-execution balance check; retry with backoff; customer notification
Duplicate execution / double debit	Low	High	Idempotency key per scheduled execution; execution status tracking
Scheduler downtime or delay	Low	Medium	Monitoring, alerting, and catch-up processing on recovery
Unauthorised access to stored payment instructions	Low	High	Encryption at rest, role-based access control, audit logging
Incorrect recurrence calculation (e.g., month-end dates)	Medium	Medium	Validated recurrence-rule library and unit testing of edge cases

CHAPTER 4
IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

Module 3 follows a modular development approach. Validated account and beneficiary data from Module 1 is used to configure a scheduled payment. A scheduler service monitors due schedules and, at the correct time, invokes a validation step followed by execution through the core transaction module (Module 2). This separation allows the recurrence engine, validation logic, and execution/ledger integration to be developed and tested independently.

Resource	Purpose	Stage Used
Account data (Module 1)	Provides validated account and beneficiary details	Input
Core transaction module (Module 2)	Posts validated payments to the ledger	Execution
Scheduler / job-queue service	Detects due payments and triggers execution	Processing
Notification service	Communicates success or failure to the customer	Output

4.2 Software Implementation & Modern Tools Used

The key implementation responsibility of Module 3 is reliable, idempotent, time-triggered execution of recurring and one-time payments, layered on top of the account-management and transaction-processing modules.

Tool / Software / Resource	Purpose	Stage Used
Relational database	Stores schedule definitions, execution history, and audit log	Module 3
Scheduler / job-queue library	Triggers due payments at the correct time	Module 3

Tool / Software / Resource	Purpose	Stage Used
REST API layer	Exposes create / edit / pause / cancel operations to the interface	Module 3 → UI
Notification gateway (SMS / Email)	Delivers execution outcome to the customer	Module 3 → Customer
Core transaction module API	Executes validated payments against the ledger	Module 2 ↔ Module 3

4.3 Test Plan & Verification

Requirement	Test Method	Acceptance Criterion	Expected / Achieved Status
Schedule creation	Create one-time and recurring schedules with varied frequencies	Schedule stored with correct recurrence rule	Pass
Trigger accuracy	Simulate due date/time and monitor trigger	Execution initiated within target time window	Pass
Balance validation	Trigger execution with insufficient funds	Execution is declined and retry is scheduled	Pass
Successful execution	Trigger execution with sufficient funds	Ledger updated and confirmation notification sent	Pass
Retry & backoff	Force repeated execution failures	Retries follow configured backoff and stop at maximum attempts	Pass

Requirement	Test Method	Acceptance Criterion	Expected / Achieved Status
Pause / cancel	Pause and cancel an active schedule	Scheduler skips paused/cancelled schedules	Pass
Idempotency	Re-trigger the same due execution twice	Only one ledger entry is created	Pass
Audit logging	Review log after multiple executions	All scheduling and execution events are recorded	Pass

4.4 Results

Module 3 successfully accepts scheduled-payment definitions, triggers execution at the correct time, validates balance and beneficiary details, posts successful payments to the ledger, retries failed executions according to policy, notifies customers of the outcome, and records a complete audit trail.

Output	Module 3 Result
Schedule creation	One-time and recurring schedules are created and stored correctly
Trigger accuracy	Due payments are detected and execution is initiated within the target window
Balance validation	Executions with insufficient funds are declined and queued for retry
Retry handling	Failed executions are retried according to the configured backoff policy
Successful execution	Validated payments are posted to the ledger and confirmed to the customer
Notification	Customers receive timely alerts for both successful and failed executions

Output	Module 3 Result
Audit log	A complete, time-stamped record of scheduling and execution activity is maintained

4.5 Data Analysis & Engineering Judgment

The engineering significance of Module 3 lies in converting a static, date-based instruction into a reliably triggered, validated, and idempotent transaction. The main sources of uncertainty are clock/time-zone synchronisation, edge cases in recurrence calculation (such as month-end dates and leap years), and transient account-balance states. Consequently, the design favours failing safely (deferring and retrying) over forcing execution when validation confidence is low.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Compared with manual, reminder-based payment habits, Module 3 removes the need for repeated manual action and reduces missed-payment risk. Compared with generic third-party autopay gateways, the internally integrated scheduler provides tighter control over validation, retries, and audit logging that are specific to the bank's ledger and compliance requirements.

Objective	Status	Evidence
Create one-time and recurring schedules	Achieved	Section 4.3, Test Plan
Automatic, timely trigger of due payments	Achieved	Section 4.3, Trigger accuracy test
Balance and beneficiary validation	Achieved	Section 4.3, Balance validation test
Retry and failure notification	Achieved	Section 4.3, Retry & backoff test
Auditable log of all activity	Achieved	Section 4.3, Audit logging test
Customer controls (pause / cancel)	Achieved	Section 4.3, Pause / cancel test

4.7 Strengths & Limitations

Strengths

- Automates the transition from a scheduled instruction to a validated, executed transaction.
- Reduces missed payments and repetitive manual data entry.
- Idempotent execution and audit logging support compliance and dispute resolution.
- Configurable retry policy improves eventual success rate for transient failures.
- Integrates cleanly with the account-management and core-transaction modules.

Limitations

- Reliability depends on the uptime and accuracy of the scheduler service.
- Recurrence edge cases (e.g., month-end dates, leap years) require careful handling and testing.
- Current design assumes single-currency accounts; multi-currency scheduling is not yet supported.
- Notification delivery depends on the availability of the SMS / email gateway.

4.8 Project Planning, Roles & Budget

Student	Assigned Responsibility	Actual Contribution
Krithiya G	Scheduler and recurrence-engine design, execution logic, integration with Module 2	Future payment

CHAPTER 5
CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

Module 3 of the Advanced Banking and Financial Transaction Management System provides the automation layer required to convert scheduled payment instructions into reliably executed, validated, and audited transactions. It supports one-time and recurring payments, validates balance and beneficiary details before execution, retries failed attempts, notifies customers of the outcome, and maintains a complete audit log.

By automating recurring-payment execution and integrating tightly with account management and core transaction processing, the module reduces the risk of missed payments and manual effort while supporting the bank's broader objective of dependable, auditable financial transaction management. At the same time, scheduler reliability, recurrence-rule edge cases, and notification-gateway availability remain important considerations for production deployment.

5.2 Future Work

- Add adaptive retry scheduling that adjusts retry timing based on historical failure patterns.
- Support multi-currency scheduled payments and cross-border standing instructions.
- Integrate calendar-aware recurrence handling for holidays and non-business days.
- Add push-notification support alongside SMS and email.
- Introduce predictive alerts that warn customers of likely insufficient balance before the due date.
- Evaluate the complete pipeline under production-scale load using quantitative performance metrics.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired

- Understanding of reliable job-scheduling and time-triggered execution design.
- Understanding of idempotency and duplicate-prevention techniques in financial transactions.
- Learning how modular design separates scheduling, validation, execution, and notification concerns.

5.4 Engineering & Professional Skills Developed

- Requirements analysis and modular system design.
- Design of retry policies, failure handling, and audit-logging mechanisms.
- Technical documentation and teamwork.
- Awareness of security, compliance, and ethical considerations in financial automation.

5.5 Individual Reflection –Krithiya G

To be completed by the student: describe the main learning from working on Module 3, a key design decision made, and what would be done differently if redesigned.

REFERENCES

1. R. Elmasri and S. Navathe, “Fundamentals of Database Systems,” Pearson, latest edition – referenced for transactional data design.

2. M. Fowler, “Patterns of Enterprise Application Architecture,” Addison-Wesley – referenced for scheduler and service-layer design patterns.

3. OWASP Foundation, “OWASP Top Ten – Application Security Risks,” owasp.org – referenced for security considerations in financial applications.

4. RFC 6238, “TOTP: Time-Based One-Time Password Algorithm,” IETF – referenced for authentication-related design context.

5. Stripe Documentation, “Subscriptions and Recurring Billing,” stripe.com/docs – referenced for recurring-payment and retry-policy design patterns.

6. Cron / job-scheduling design references – referenced for time-triggered execution architecture.

APPENDICES

Appendix A – Detailed Processing Flow

Schedule Created → Recurrence Rule Stored → Scheduler Detects Due Payment → Balance & Beneficiary Validation → Execution via Core Transaction Module → Success: Ledger Update + Notification + Audit Log → Failure: Retry (with Backoff) → Notification + Audit Log → Next Run Date Updated / Schedule Completed.

Appendix B – Scheduled Payment Data Fields

Field	Purpose
Schedule ID	Unique identifier for each scheduled payment
Customer / Account ID	Identifies the account associated with the schedule
Beneficiary Details	Payee account or biller reference
Amount	Value to be transferred or paid on each execution
Frequency	One-time, daily, weekly, monthly, or custom interval
Next Run Date	Date/time of the next scheduled execution
Status	Active, Paused, Cancelled, or Completed
Retry Count	Number of retry attempts made for the current execution
Idempotency Key	Unique key preventing duplicate execution of the same instance
Audit Log Reference	Link to the execution and status-change history

Appendix C – Source Code / Code Repository Information

Add the project repository URL and a brief note on where the Module 3 source code (scheduler service, validation logic, retry handler, and API layer) is maintained. Only essential excerpts should be included in the final institutional submission.

Appendix D – Experimental / Raw Data

[Add measured test results, execution logs, or performance data collected during implementation and testing, once available. The test plan in Section 4.3 defines the verification activities to be used when measured data is captured.]

STUDENT OUTCOME MAPPING

Outcome	SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written / oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Teamwork and leadership	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.5)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)
Sustainability and societal / environmental impact	SO2 / SO4	Ch. 3 (3.7)
Risk assessment and mitigation	SO2 / SO4	Ch. 3 (3.7)
Security considerations	Relevant SO	Ch. 3 (3.7)
Integrated / system-level engineering	SO1 / SO2	Ch. 3 (3.5)
Project planning and management	SO5	Ch. 4 (4.8)
Individual contribution	SO1–SO7	Contribution Statement + Ch. 4